

Exercises 4

Exercise 1 (First-Order Formulas). Among the following formulas of first order, classical logic, four are theorems and two are not. Prove two of them using usual pen-and-paper (but still reasonably formal) mathematical proofs, and two using formal resolution proofs. For the two that are not theorems, produce models (base set and interpretation) where they do not hold.

1. $(\exists y. \forall x. P(x, y)) \rightarrow \forall x. \exists y. P(x, y)$
2. $\exists x. \forall y. (Q(x) \rightarrow Q(y))$
3. $(\neg \forall x. \exists y. P(x, f(y))) \vee \exists y. \forall x. P(f(y), x)$
4. $(\forall x, y, z. P(x, y) \wedge P(y, z) \rightarrow P(x, z)) \wedge (\forall x, y. P(x, y) \vee P(y, x)) \rightarrow \forall x. P(x, x)$
5. $(\exists x. (Q(x) \wedge Q(f(x)))) \rightarrow (\forall y. Q(y) \vee Q(f(y)))$
6. $\forall x. \forall y. P(x, y) \rightarrow \forall x. \forall y. P(y, x)$

Exercise 2 (Weakest Precondition). Recall the definition of the weakest precondition operator:

$$\text{wp}(r, Q) = \{s \mid \forall s'. (s, s') \in r \rightarrow s' \in Q\}$$

1. Prove or disprove the following properties:

- (a) $\text{wp}(r_1 \cup r_2, Q) = \text{wp}(r_1, Q) \cup \text{wp}(r_2, Q)$
- (b) $\text{wp}(r_1 \cup r_2, Q) = \text{wp}(r_1, Q) \cap \text{wp}(r_2, Q)$
- (c) $\text{wp}(r, Q_1 \cap Q_2) = \text{wp}(r, Q_1) \cap \text{wp}(r, Q_2)$
- (d) $\text{wp}(r, Q_1 \cup Q_2) = \text{wp}(r, Q_1) \cup \text{wp}(r, Q_2)$

For those that are false, do any of them hold if r , r_1 , and r_2 are restricted to being a functional relation? A relation r is functional if it satisfies

$$\forall x, y_1, y_2. ((x, y_1) \in r \wedge (x, y_2) \in r) \rightarrow (y_1 = y_2) .$$

2. Let $r \subseteq S \times S$ and $Q \subseteq S$. Give an expression defining weakest precondition $\text{wp}(r, Q)$ using the operations inverse of a relation, $(\cdot^{-1})$, set difference $(\cdot \setminus \cdot)$, image of a set under a relation, $(\cdot [\cdot])$, and complement w.r.t S , $\cdot^c (= S \setminus \cdot)$. Prove that your expression is correct

by expanding the definitions of `wp` as well as of the relational and set operations.

Exercise 3 (Programing With Integers). Consider the following program:

```

1 import stainless.lang.*
2 type INT = BigInt
3 case class Container(var x: INT, var y: INT):
4   def fun: Unit = {
5     require(x > 0 && y > 0)
6     if x > y then
7       x = x + y
8       y = x - y
9       x = x - y
10    else
11      y = y - x
12  }.ensuring(2 * old(this).x + old(this).y > 2 * x + y &&
13            x >= 0 && y >= 0)

```

Take I as the set representing values of type `INT`.

1. Compute $\rho(\text{fun}) \subseteq I^4$, the relation corresponding to the imperative program represented by `fun`, relating the previous state (x, y) to the final state (x', y') , by expressing all intermediate formulas corresponding to subprograms.
2. Write the formula expressing the correctness of the ensuring clause.
3. Is the verification condition formula valid when `INT` denotes mathematical integers with their usual operations (`type INT = BigInt` in Scala), i.e. when $I = \mathbb{Z}$?
4. Is the verification condition formula valid when `INT` denotes 32-bit machine integers and where operations are interpreted as the usual machine operations (`type INT = Int` in Scala), i.e. when $I = \mathbb{Z}_{2^{32}}$?

$$\mathbb{Z}_{2^{32}} = \{-2^{31}, -2^{31} + 1, \dots, -1, 0, 1, \dots, 2^{31} - 1\}$$

Exercise 4 (Hoare Logic Proof). Give a complete Hoare logic proof for the following program:

```

{n >= 0 && d > 0}
  q = 0
  r = n
  while ( r >= d ) {
    q = q + 1
    r = r - d
  }
{n == q * d + r && 0 <= r < d}

```

The proof should include step-by-step annotation for each line of the program, as in the example proof in the lecture.

Exercise 5 (Iterating a Relation). Let $M = (S, I, r, A)$ be a transition system and $\bar{r} = \{(s, s') \mid (s, a, s') \in r\}$, as usual. Let $\Delta = \{(x, x) \mid x \in S\}$.

Let $\bar{r}^k$ denote the usual composition of relation $\bar{r}$ with itself k times.

Define the sequence of relations r_n , for all non-negative integers n , as follows:

- $r_0 = \Delta \cup \bar{r}$
 - $r_{n+1} = r_n \circ r_n$
1. Prove that $r_n \subseteq r_{n+1}$ for every n .
 2. Prove that for every n and every k where $0 \leq k \leq 2^n$ we have $\bar{r}^k \subseteq r_n$.
 3. Suppose that S is finite. Find a bound B as a function of $|S|$ such that

$$\text{Reach}(M) \subseteq r_B[I]$$

Aim to find as small bound as possible.

Exercise 6 (Approximating Relations). Consider a guarded command language operating on a state. The set of all states is denoted by U . The language contains the following commands, constructed inductively:

- atomic commands $\{a_1, a_2, \dots\}$, representing arbitrary user-defined state transitions
- non-deterministic choice: **if** (*) c_1 **else** c_2
- sequential composition: $c_1; c_2$
- assume statement: **assume** G , where the guard G is a predicate on states

where c, c_1, c_2 denote commands.

We interpret programs in this language as relations on states ($\subseteq U \times U$), relating the input state to possible output states.

Corresponding to each program, we build an expression denoting its characteristic relation. The language of relations is constructed inductively:

- atomic relations $r_1, r_2, \dots \subseteq U \times U$
- diagonal relations $\Delta_P = \{(x, x) \mid x \in P\}$ for $P \subseteq U$
- union of relations $(\cdot \cup \cdot)$
- composition of relations $(\cdot \circ \cdot)$

We can define the translation from commands to relational expressions (ρ) as follows:

c	$\rho(c)$
a_i	r_i
if (*) c_1 else c_2	$\rho(c_1) \cup \rho(c_2)$
$c_1 ; c_2$	$\rho(c_1) \circ \rho(c_2)$
assume G	$\Delta_{S(G)}$

where $\Delta_{S(G)}$ denotes the diagonal relation for the set of states satisfying G , i.e. Δ_P where $P = \{s \in U \mid G(s)\}$.

The atomic relations r_i can be constructed for each atomic command. If the state consists of two integer variables, i.e., if $U = \mathbb{Z} \times \mathbb{Z}$, then the atomic command $\mathbf{x} := \mathbf{y} + 1$, for example, can be represented by the relation

$$r((x, y), (x', y')) \iff x' = y + 1 \wedge y' = y .$$

By $E(a_1, \dots, a_n)$ we denote an arbitrary relational expression built from some atomic relations $a_1, \dots, a_n$.

Let us call a relation $e \subseteq U \times U$ an *effect* if it is reflexive and transitive. We say that an effect e *approximates* an expression $E(a_1, \dots, a_n)$ if $E(a_1, \dots, a_n) \subseteq e$.

1. Assuming $U = \mathbb{Z} \times \mathbb{Z}$, for the following atomic commands, compute the atomic relations corresponding to them:

(a) $\mathbf{x} := \mathbf{x} + 1$

(b) $\mathbf{y} := \mathbf{y} + \mathbf{x}$

(c) $\mathbf{x} := \mathbf{x}$

(d) **havoc**($\mathbf{x}$) (non-deterministically assign $\mathbf{x}$)

2. Compute the relation $\rho(C)$ for the following program C :

$$\mathbf{y} := \mathbf{x} + \mathbf{y} ; \mathbf{x} := \mathbf{y} - \mathbf{x} ; \mathbf{y} := \mathbf{y} - \mathbf{x}$$

3. Prove that if a relation e is an effect and $r_i \subseteq e$ for all $1 \leq i \leq n$, then

$$\forall E. E(a_1, \dots, a_n) \subseteq e$$

which is to say that if e approximates a set of atomic relations, then it approximates any expression built from those relations.

4. For each of the following representations of relations, prove or disprove whether they are effects. Assume as before that $U = \mathbb{Z} \times \mathbb{Z}$, with a state having two variables x and y . Further, informally, what can we say about programs whose relations are approximated by these effects?
 - (a) $\rho(\text{havoc}(x))$
 - (b) Δ_U
 - (c) $\{((x, y), (x', y')) \mid y \geq 0 \rightarrow (x' \leq x \wedge y' \geq 0)\}$
5. For each of the following programs, argue whether they are approximated by each of the effects you identified in the previous problem:
 - (a) `x := x + y ; y = x - y ; x := x - y`
 - (b) `if (*) x := x - 2 else x := x - 1`
 - (c) `assume y > 2 ; x := x - 1 ; y := y - 2`
 - (d) `assume y > 0`